

Preguntas preparación parcial

Propiedades de los tensores

- Dado el tensor `x = torch.tensor([[1, 2], [3, 4]])`, ¿cuál es su `dtype`, `device`, `ndim` y `shape`?
- Explica la utilidad de la propiedad `device` en PyTorch. Proporciona un ejemplo para mover un tensor a la GPU.
- Crea un tensor unidimensional con 5 elementos y muestra cómo consultar su número de dimensiones y su forma.

Concepto de Broadcasting

- Explica el concepto de broadcasting en PyTorch. ¿Por qué es útil?
- Dado `x = torch.tensor([[1, 2], [3, 4]])` y `y = torch.tensor([10, 20])`, ¿qué sucederá al realizar `z = x + y`? Especifica la forma y los valores de `z`.
- ¿Es posible sumar un tensor con forma `(4, 3)` y otro con forma `(3,)`? ¿Por qué?

Slicing en tensores

- Dado el tensor `x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`, escribe el código para seleccionar la columna segunda columna.
- Extrae las dos primeras filas y las dos primeras columnas del tensor `x` con forma `(3, 3)`.
- Usa slicing para seleccionar los elementos de las posiciones impares en un tensor unidimensional.

Diferencias entre cat y stack

- ¿Cuál es la principal diferencia entre `torch.cat` y `torch.stack`? Proporciona un ejemplo.
- Dado `x = torch.tensor([1, 2])` y `y = torch.tensor([3, 4])`, escribe el código para concatenarlos a lo largo de una nueva dimensión.
- ¿Qué hará `torch.stack([x, y], dim=0)` si `x` e `y` tienen forma `(2,)`? Proporciona el resultado.

Funciones squeeze, unsqueeze, reshape

- Dado un tensor con forma `(1, 3, 1)`, ¿cómo eliminarías todas las dimensiones de tamaño 1?

- ¿Qué hace `torch.unsqueeze(x, dim=0)`? Explica y proporciona un ejemplo.
- ¿Cómo reestructurarías un tensor de forma `(2, 3)` a `(6,)` usando `reshape`?

Uso de GPU

- Explica cómo verificar si una GPU está disponible en PyTorch.
- Escribe el código para mover un tensor `x` a la GPU y luego devolverlo a la CPU.
- ¿Qué sucede si intentas operar entre un tensor en GPU y otro en CPU? Proporciona una solución al problema.

Datasets y DataLoaders

- ¿Qué funciones debemos implementar cuando creamos un `DataSet` personalizado? Para que sirven?
- Explica cómo usar el parámetro `batch_size` en un `DataLoader` y qué impacto tiene en el entrenamiento. Muestre alguna situación que convenga subirlo/bajarlo.
- ¿Qué hace el parámetro `shuffle` en un `DataLoader` y cuándo es útil activarlo?
- Dado el siguiente código:

```
dataloader = DataLoader(dataset, batch_size=4, num_workers=2)
```

¿Qué significa `num_workers` y cómo afecta el rendimiento del entrenamiento?

Train & Eval

- ¿Cuál es la diferencia entre `model.train()` y `model.eval()` en PyTorch?
- ¿Qué sucede con las capas de dropout cuando llamas a `model.eval()`?
- Dado el siguiente código:

```
model.eval()  
with torch.no_grad():  
    output = model(x)
```

¿Por qué se utiliza `torch.no_grad()` en este caso?

- ¿Qué problemas podrían surgir si olvidas cambiar a `model.eval()` durante la evaluación?

Pérdidas (train loss, val loss)

- ¿Qué representa la `train loss` y la `val loss` en un modelo de machine learning?

- Si la `train loss` disminuye pero la `val loss` aumenta, ¿qué problema podría estar ocurriendo? Cómo se puede detener este fenómeno?

Bucles de entrenamiento y evaluación

- Dado el siguiente bucle:

```
for epoch in range(epochs):  
    model.train()  
    for batch in train_loader:  
        ...  
    model.eval()  
    with torch.no_grad():  
        for batch in val_loader:  
            ...
```

Explica brevemente qué hace cada línea del código.

Weights & Biases (WandB)

- ¿Qué hace Weights & Biases y por qué es útil en el entrenamiento de modelos?
- Explique la diferencia entre un `sweep` y un `run`.

Capa `nn.Linear`

- Explica qué hace la capa `nn.Linear` y cómo transforma una entrada en una salida.
- Dado el siguiente código:

```
layer = nn.Linear(4, 2)  
x = torch.randn(3, 4)  
output = layer(x)
```

¿Cuál será la forma de `output` y qué representa cada dimensión?

- ¿Qué impacto tiene el uso de `bias=True` al definir una capa `nn.Linear`?

Capa `nn.Dropout`

- ¿Cuál es el propósito de la capa `nn.Dropout` y cómo afecta el entrenamiento de un modelo?

- Dado el siguiente código:

```
dropout = nn.Dropout(p=0.5)
x = torch.tensor([1.0, 2.0, 3.0])
output = dropout(x)
```

Explica qué valores podría tomar `output`.

Capa `nn.Embedding`

- ¿Qué representa la capa `nn.Embedding` y para qué tipo de datos es útil?
- Dado el siguiente código:

```
embedding = nn.Embedding(6, 3)
input = torch.tensor([0, 2, 4])
output = embedding(input)
```

¿Cuál será la shape de `output` y qué representa cada dimensión?

- ¿Cómo inicializarías un embedding con pesos preentrenados?

**Capas convolucionales

- Dado el siguiente código:

```
conv = nn.Conv2d(in_channels=1, out_channels=3, kernel_size=3, stride=1,
padding=1)
x = torch.randn(1, 1, 28, 28)
output = conv(x)
```

¿Qué shape tiene output? ¿Y si `stride` es 0?

Capas de pooling

- ¿Qué es una capa de pooling y cuál es su propósito en una red convolucional?
- Dado el siguiente código:

```
pool = nn.MaxPool2d(kernel_size=2, stride=2)
x = torch.randn(8, 3, 32, 32)
output = pool(x)
```

¿Cuál será la forma del tensor `output`?

Capas recurrentes (`nn.RNN`, `nn.LSTM`, `nn.GRU`)

- Explica las principales diferencias entre `nn.RNN` y `nn.LSTM`.
- Dado el siguiente código:

```
rnn = nn.RNN(input_size=10, hidden_size=20, num_layers=2, batch_first=True)
x = torch.randn(5, 15, 10)
output, h_n = rnn(x)
```

¿Cuál será la forma de `output` y `h_n`?

Preguntas sobre DenseNet

- Explica el concepto de conexión densa en DenseNet. ¿Cómo se diferencian de las conexiones residuales en ResNet?
 - ¿Qué representa el parámetro `growth_rate` en una DenseNet?
 - ¿Qué beneficio aporta la concatenación de características en lugar de su suma como en ResNet?
-

Regularización

- ¿Qué es la regularización en el contexto de redes neuronales y por qué es importante?
 - Nombre alguna de las técnicas vistas en clase y la idea detrás de ellas.
-

Data Augmentation

- ¿Qué es data augmentation y cómo ayuda a mejorar el desempeño de un modelo de aprendizaje profundo?
- Proporciona ejemplos comunes de data augmentation para imágenes.
- Proporcionar un ejemplo donde es contraproducente proporcionar una determinada transformación.
- Dado el siguiente código para data augmentation en imágenes:

```
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(30),
    transforms.ToImage()
])
```

Explica qué hace cada transformación y sus parámetros. ¿Es posible que la imagen visualmente salga igual de cómo entró?

Early Stopping

- ¿Qué es el early stopping y cómo puede prevenir el sobreentrenamiento?
 - Describe cómo la loss en validación (`val_loss`) se utiliza para implementar el early stopping.
-

Preguntas sobre pre-procesamiento y vocabulario

Preprocesamiento

- ¿Cuál es el objetivo de normalizar texto antes de entrenar un modelo NLP? Proporciona un ejemplo práctico.
- ¿Cuáles son las transformaciones que aplicarías a un texto para analizar su sentimiento?

Vocabulario

- ¿Qué es un token y cómo se relaciona con un vocabulario en NLP?
- Explica el propósito de limitar el tamaño del vocabulario (`max_vocab_size`) en datasets grandes. ¿Cuáles descartarías?
- Dado el siguiente vocabulario:

```
vocab = {"<pad>": 0, "<unk>": 1, "el": 2, "análisis": 3, "texto": 4}
```

¿Cómo se representaría la frase "el análisis de texto" usando este vocabulario?

Padding y truncamiento

- Explica el propósito del padding y truncamiento en NLP.

Representación numérica de texto

- ¿Por qué no podemos usar directamente palabras en una red neuronal?
- Explica cómo se utiliza `nn.Embedding` para mapear palabras a vectores densos.
- Dado un embedding:

```
embedding = nn.Embedding(10, 4)
input = torch.tensor([[0, 1, 2]])
```

¿Cuál será la forma de la salida y qué representa cada dimensión?

Seq2Seq

- Explica cómo funcionan el codificador (encoder) y el decodificador (decoder) en un modelo Seq2Seq. ¿Qué información pasa del primero al segundo?
 - ¿Qué significa **Teacher Forcing** y cómo afecta al entrenamiento?
 - ¿Por qué es útil agregar un token **<SOS>** al inicio de una secuencia en el decodificador?
 - Durante la inferencia, ¿cómo se determina el fin de una predicción en un modelo Seq2Seq?
-

Transformers

- ¿Cuántos mecanismos de atención hay en el encoder y decoder del transformer del paper "attention is all you need"? ¿En qué se diferencian?
- Dado el cálculo de atención:

```
scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
attention = torch.softmax(scores, dim=-1)
context = torch.matmul(attention, value)
```

Explica qué hace cada línea.

- Dado el siguiente fragmento:

```
pos_encoding = torch.sin(position / (10000 ** (2 * (i // 2) / d_model)))
```

¿Cómo ayuda este cálculo a incorporar información posicional en un Transformer?

- Explica cómo las máscaras evitan que un Transformer preste atención a posiciones no deseadas durante el entrenamiento.